

Cloud Project

on

"REAL-TIME EMPLOYEE CHAT & ANNOUNCEMENTS PLATFORM"

Submitted by:

Ojasvi Mohapatra

Alfin Shaji

Mohd Aatif Sayyed

Abdulharis Shaikh

Mohamed reefath

1.

Introduction

Modern organizations depend on fast, reliable internal communication. Traditional tools such as email are asynchronous and ill-suited for real-time collaboration, while commercial messaging solutions like Slack often carry prohibitive licensing costs and store sensitive employee data on third-party servers. This project addresses those gaps by proposing a cloud-native, serverless messaging platform purpose-built for organizational use.

The Real-Time Employee Chat & Announcements Platform is a Slack-lite internal communication system built entirely on AWS serverless technologies. Employees can exchange direct messages, post in department channels, and instantly receive company-wide announcements broadcast by HR — all without any server infrastructure to provision or manage.

The platform leverages the API Gateway WebSocket API to maintain persistent, bidirectional connections between clients and the cloud backend. AWS Lambda functions handle all business logic, Amazon DynamoDB stores connection state and message history, Amazon Cognito provides authenticated access, and Amazon SNS together with Amazon SES powers the HR broadcast pipeline. The static frontend is hosted on Amazon S3 and delivered through Amazon CloudFront.

The system automatically:

- Establishes and manages persistent WebSocket connections authenticated via Cognito JWT tokens.
- Routes direct messages and channel messages to all currently connected recipients in real time.
- Serves message history (last 50 messages per channel) to employees on reconnection.
- Broadcasts HR announcements simultaneously over WebSocket and email (SES) via SNS fan-out.
- Applies priority styling (normal / urgent) to announcements on the frontend.
- Auto-expires stale connection records using DynamoDB TTL, keeping the connections table clean.

2. Objectives

The primary goals of this project are:

Implement Real-Time WebSocket Communication

Use API Gateway WebSocket API with Lambda-backed routes (\$connect, \$disconnect, sendMessage) to enable persistent, low-latency bidirectional messaging without polling.

Enable Channel and Direct Messaging

Support department channels (e.g., #engineering, #hr) and one-to-one direct messages, with history loaded from DynamoDB on reconnection.

Build an HR Broadcast Pipeline

Allow HR admins to post company-wide announcements that are instantly delivered to all active WebSocket connections and emailed to all employees through SNS and SES.

Develop a Recruiter-Friendly Frontend

Create a single-page HTML/JS application hosted on S3 with a sidebar for channels and DMs, a chat window, an announcement panel, and automatic WebSocket reconnection logic.

Ensure Security and Scalability

Secure all connections with Cognito JWT validation, enforce least-privilege IAM roles per Lambda, serve all traffic over WSS/HTTPS, and leverage serverless auto-scaling to handle variable organizational load.

3.

System Architecture

The platform is organized around two main paths — the Message Processing Path and the HR Broadcast Path — with a shared DynamoDB backend and a CloudFront-fronted S3 static site.

High-level flow:

Client (Browser) → CloudFront / S3 → Cognito → API Gateway WebSocket → Lambda → DynamoDB

HR Admin → SNS Topic → (1) SES → Employee Email / (2) Lambda → All Active WebSockets

3.1 Message Processing Path

When an employee opens the chat application, the browser establishes a WebSocket connection to API Gateway, passing the Cognito JWT as a query parameter. The \$connect Lambda validates the token, then writes the connectionId, employee_id, and channel to the DynamoDB connections table with a 24-hour TTL. On sendMessage, the routing Lambda queries DynamoDB for all active connectionIds in the target channel and calls the API Gateway Management API to push the payload to each recipient. On disconnect, the \$disconnect Lambda deletes the connection record.

3.2 HR Broadcast Path

When an HR admin submits an announcement, the chat-announcement-publish Lambda publishes a JSON message (containing the announcement text and priority flag) to an SNS topic. SNS fans the message out to two subscriptions: an SES subscription that triggers an email to all registered employees, and a Lambda subscription (broadcast_announcement) that iterates all active connectionIds in DynamoDB and pushes the announcement over WebSocket to every connected client.

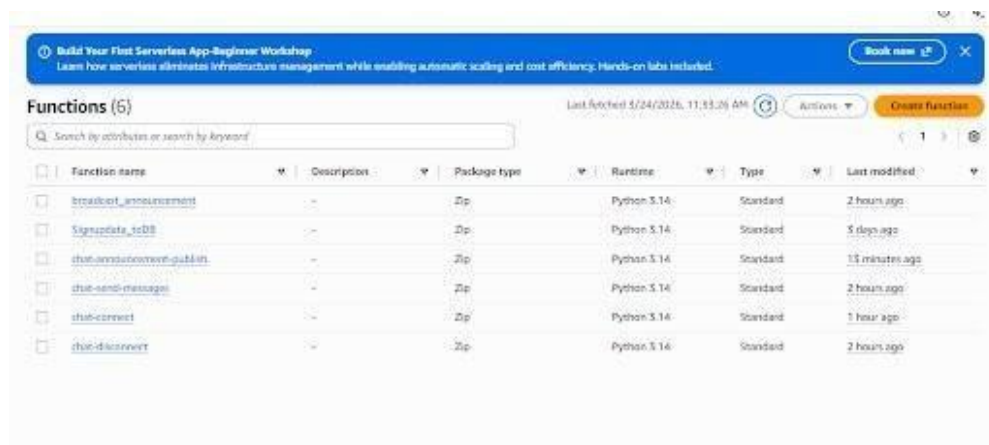


Figure 1: AWS Lambda console showing all six deployed functions for the platform.

4.

WebSocket Connection & Messaging

4.1 Connection Lifecycle

API Gateway WebSocket API manages persistent connections through three dedicated routes:

- **\$connect** — Invokes the chat-connect Lambda. The function decodes and verifies the Cognito JWT from the query string. On success, it writes a record to the DynamoDB connections table: `connectionId` (PK), `employee_id`, `channel`, `connected_at`, and a TTL set to `current_time + 86400` seconds (24 hours).
- **\$disconnect** — Invokes the chat-disconnect Lambda, which deletes the connection record from DynamoDB, immediately removing stale entries without waiting for TTL expiry.
- **sendMessage** — Invokes the chat-send-messages Lambda. The function reads the target channel or DM recipient from the message payload, queries DynamoDB for all matching active `connectionIds`, and calls the API Gateway Management API `postToConnection` endpoint for each one.

4.2 DynamoDB Schema

Two tables underpin the platform:

Table	Primary Key	Sort Key	Key Attributes
connections	connectionId (PK)	—	employee_id, channel, connected_at,
messages	message_id (PK)	channel_id (SK)	sender_id, content, timestamp

TTL

The connections table uses a 24-hour TTL so that records for employees who close their browser without triggering `$disconnect` are automatically purged by DynamoDB, preventing ghost entries from accumulating and ensuring the broadcast Lambda never attempts delivery to expired connections. The messages table is append-only and capped at the last 50 messages per channel to remain within free-tier DynamoDB read limits.

5.

4.3 Message History on Reconnect

When a client reconnects, the `$connect` Lambda performs a paginated query on the messages table filtered by `channel_id`, sorted by timestamp descending, limited to 50 items. The history payload is pushed back over the newly established WebSocket connection so the employee sees recent context immediately without any additional client-side requests.

Channels & Direct Messaging

The platform supports two channel types. Department channels such as `#open-channel` and `#hr` allow all employees assigned to that channel to broadcast messages to each other. Direct Messages (DMs) are one-to-one conversations keyed by a `channel_id` composed of the two employee IDs in sorted order, ensuring the same DM thread is referenced regardless of which participant initiates it.

The frontend sidebar lists all available company channels and the employee's DM threads. Clicking a channel loads history and switches the active WebSocket routing context. The "+ New DM" button allows the employee to open a conversation with any other employee by username.

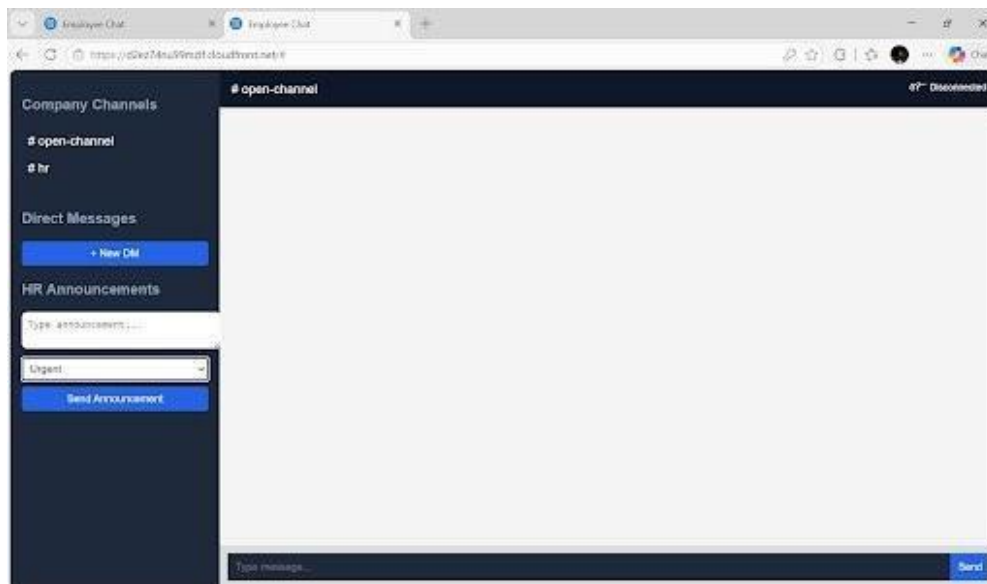


Figure 2: Frontend showing the sidebar with Company Channels, Direct Messages, and the HR Announcements panel (disconnected state).

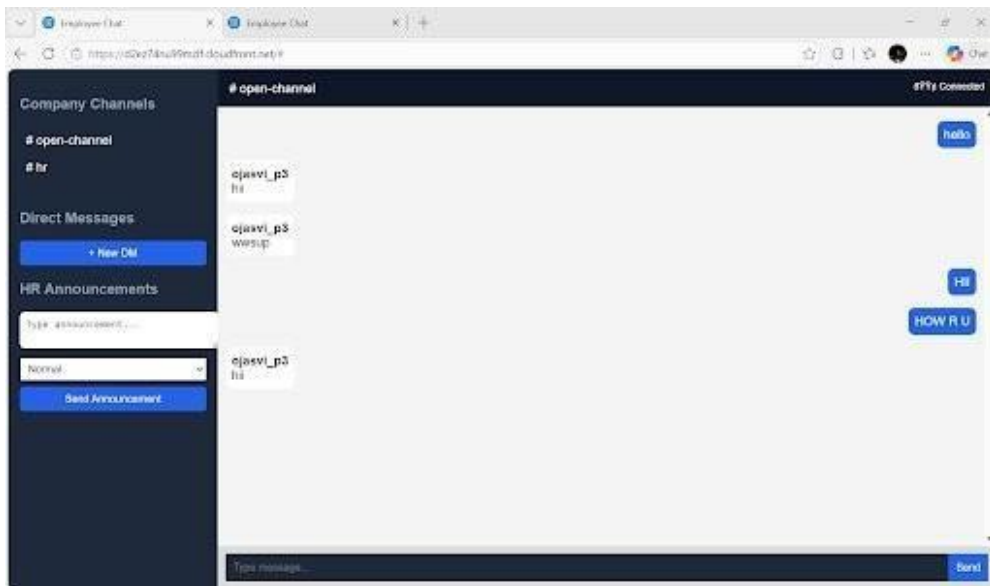


Figure 3: Two employees exchanging real-time messages in #open-channel — messages appear instantly on both sides.

6. HR Announcements Broadcast

HR admins can post company-wide announcements through a dedicated panel in the sidebar. The admin types the announcement text, selects a priority (Normal or Urgent), and clicks Send Announcement. This triggers the chat-announcement-publish Lambda, which publishes a structured JSON payload to an SNS topic:

```
{"type": "hr_announcement", "announcement": "<text>", "priority": "urgent"}
```

SNS fans the message out to two targets simultaneously:

- **SES Email Subscription** — All registered employees receive an email from AWS Notifications containing the announcement payload. The email subject line reflects the priority: 'Company Announcement: URGENT' for urgent messages.
- **broadcast_announcement Lambda** — The Lambda scans the DynamoDB connections table for all active connectionIds and calls the API Gateway Management API to push the announcement to every connected client in real time. The priority flag drives visual styling on the frontend (e.g., red banner for urgent).

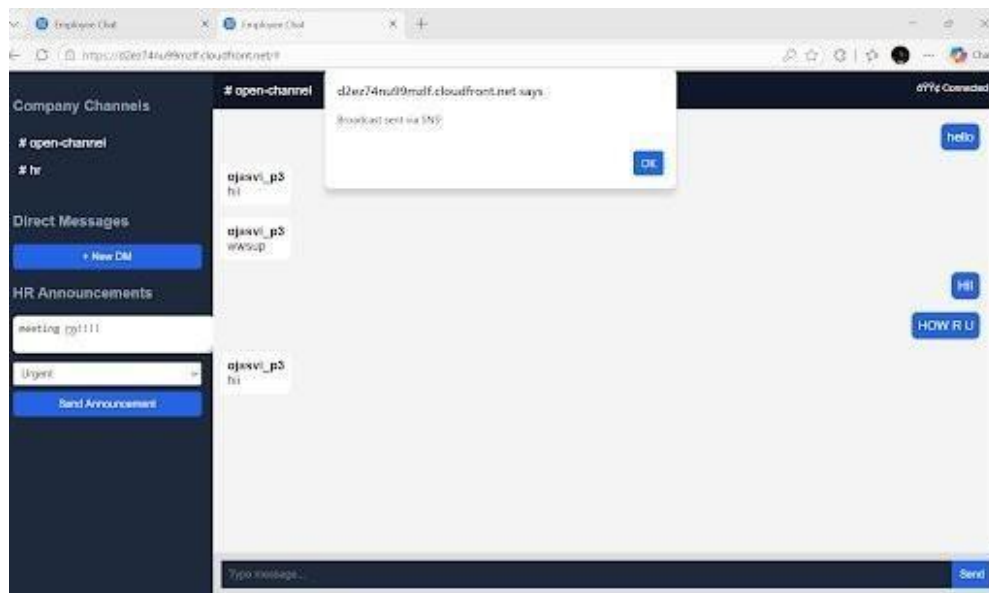


Figure 4: Browser alert confirming that the HR announcement was broadcast via SNS to all active WebSocket connections.

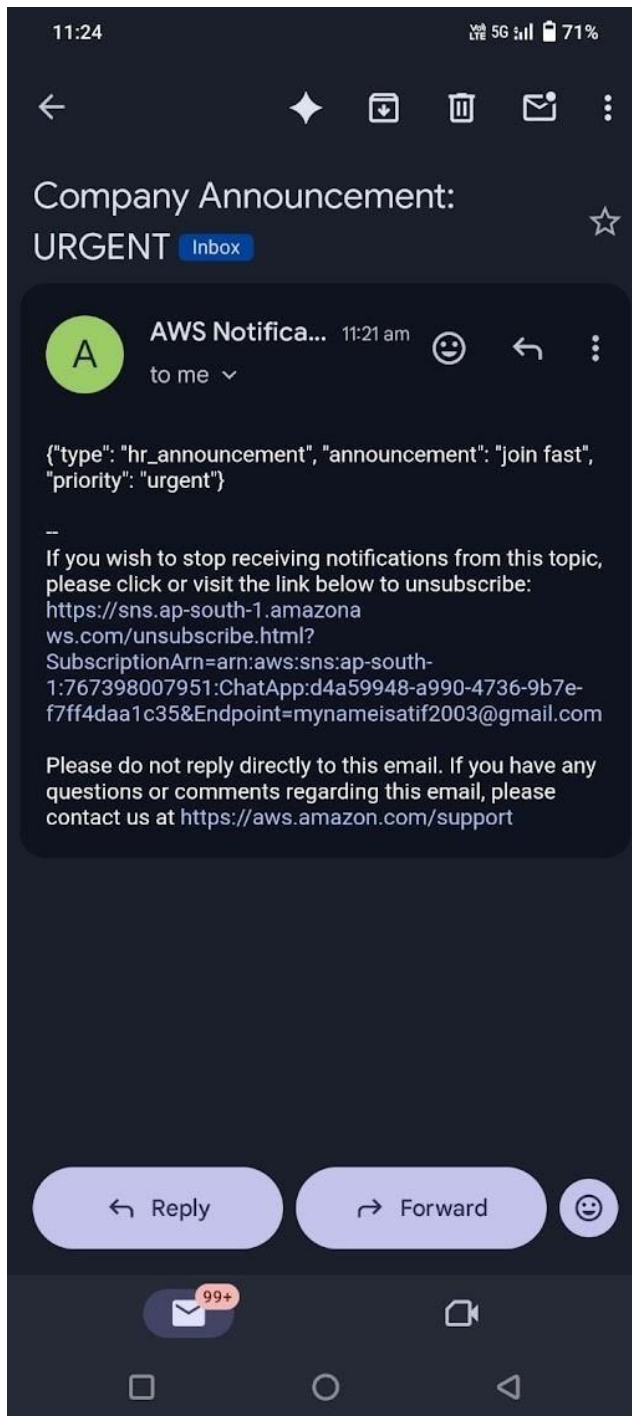


Figure 5: Mobile email inbox showing the URGENT company announcement delivered via SES through the SNS subscription.

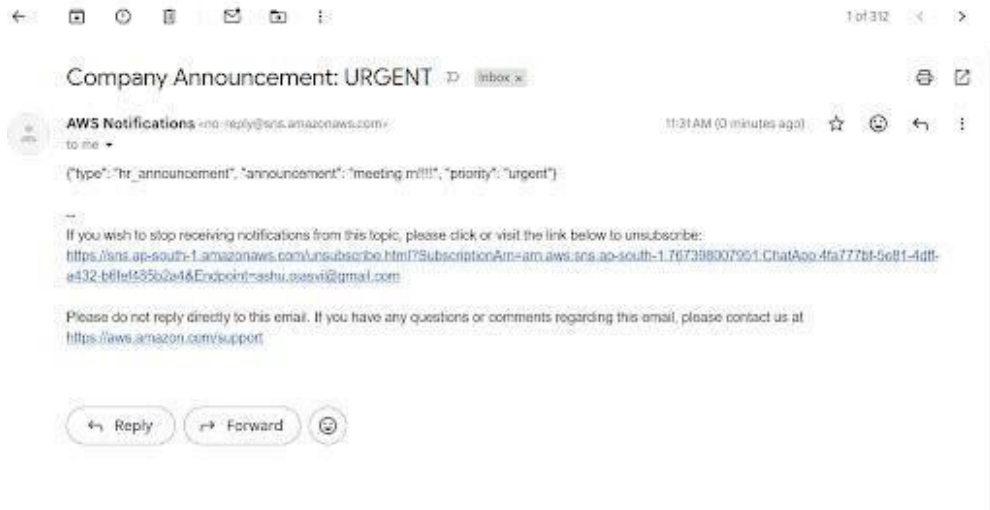


Figure 6: Gmail inbox showing a second urgent announcement email ('meeting m!!!!') delivered to a different employee's account.

7. Frontend

The frontend is a single-page application built with HTML, CSS, and vanilla JavaScript. The static files are hosted on an S3 bucket with static website hosting enabled and served globally through Amazon CloudFront over HTTPS.

Key frontend features:

- Sidebar listing all company channels and direct message threads.
- Chat window displaying message history with sender name and timestamp.
- Input box for composing and sending messages to the active channel.
- HR Announcements panel with a text area and a priority dropdown (Normal / Urgent).
- WebSocket connection established on login using the Cognito JWT appended as a query parameter to the WSS connection URL.
- Automatic reconnection logic: if the WebSocket drops, the client retries every 3 seconds with exponential back-off to restore the session transparently.
- Connection status indicator ('Connected' / 'Disconnected') displayed in the top-right of the chat header.

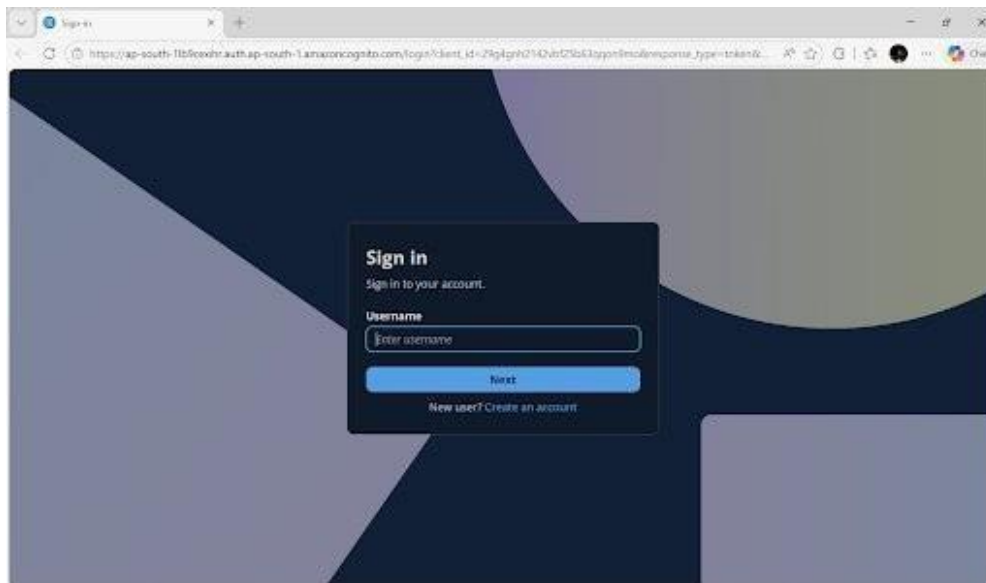


Figure 7: Amazon Cognito hosted UI sign-in page — employees authenticate here before the WebSocket connection is established.

8. Security

8.1 Authentication

Amazon Cognito provides user pool-based authentication. On successful login, Cognito issues a JWT (ID token). The frontend appends this token as a query parameter when initiating the WebSocket handshake. The \$connect Lambda decodes and verifies the JWT signature against the Cognito User Pool's public keys. Any connection attempt with a

missing, expired, or tampered token is rejected with a 401 response before a connectionId is ever stored in DynamoDB.

8.2 IAM Least Privilege

Each Lambda function is bound to a dedicated IAM execution role that grants only the minimum permissions required. The chat-connect and chat-disconnect Lambdas have write access only to the connections table. The chat-send-messages Lambda has read access to the connections table and execute-api permission on the Management API. The broadcast_announcement Lambda has read access to the connections table and execute-api permissions. No Lambda has unrestricted DynamoDB or API Gateway access.

8.3 Transport Security

All WebSocket traffic uses WSS (WebSocket Secure), which runs over TLS. The S3 static site is served exclusively over HTTPS via CloudFront, with HTTP redirected to HTTPS. CORS headers on the API Gateway are configured to accept requests only from the CloudFront distribution origin.

9. Implementation Status

The Real-Time Employee Chat & Announcements Platform has been successfully implemented end-to-end. The system is fully functional and demonstrates a complete serverless real-time communication workflow on AWS. The following components are fully operational:

- Amazon Cognito User Pool with hosted sign-in UI
- API Gateway WebSocket API with \$connect, \$disconnect, and sendMessage routes
- chat-connect Lambda: JWT validation and DynamoDB connection registration with 24-hour TTL
- chat-disconnect Lambda: connection record deletion on session end
- chat-send-messages Lambda: real-time message routing to all active channel connections
- chat-announcement-publish Lambda: SNS topic publish with priority flag
- broadcast_announcement Lambda: WebSocket push to all active connections on SNS trigger
- SignupUsers_toDB Lambda: user registration persistence
- DynamoDB connections table with TTL auto-expiry
- DynamoDB messages table with append-only message history (capped at 50 per channel)
- SNS topic with dual subscriptions: SES email and broadcast Lambda
- SES email delivery of HR announcements to all registered employees
- S3 static site hosting with CloudFront HTTPS distribution

- Reconnection logic with 3-second retry and exponential back-off
- Priority-based styling for Normal and Urgent announcements
- CSV report download option on the dashboard
- IAM least-privilege roles for all Lambda functions
- CORS configuration for frontend origin access

10. Minor Limitations

Although the system is fully functional, a few areas can be improved to further harden it for large-scale enterprise use:

1. Direct Scan for Broadcast

The broadcast_announcement Lambda currently scans the entire connections table to find all active connectionIds. For large organizations with many simultaneous connections, a Global Secondary Index (GSI) on the channel or a dedicated broadcast-connections index would reduce read consumption.

2. No Presence / Typing Indicators

The current implementation does not support presence detection (online/offline badges) or typing indicators. These could be added with lightweight heartbeat messages and a separate presence table in DynamoDB.

3. Email Formatting

SES currently delivers the raw JSON payload as the email body. A Lambda transformation step or an SES template could render a well-formatted HTML email with the announcement text, priority badge, and company branding.

4. High-Volume Upload Handling

S3 triggers Lambda directly for file-based operations. For extremely large-scale enterprise deployments, Amazon SQS could be introduced between S3 and Lambda to queue requests and enable batch processing and failure retries.

11. Future Enhancements

Although the system is fully functional, the following improvements can further enhance its intelligence, scalability, and production readiness:

1. Push Notifications (Mobile)

Integrate Amazon SNS mobile push to deliver messages and announcements to employees on iOS and Android devices even when the browser tab is closed.

2. Message Reactions and Threading

Add emoji reactions and threaded replies to channel messages, stored as nested items in the messages table using a composite sort key.

3. File and Image Sharing

Allow employees to upload attachments to S3 directly from the chat window using pre-signed URLs, with the file link broadcast via the existing WebSocket pipeline.

4. Full Authentication & Role-Based Access

Extend Cognito with groups to enforce role-based access: only users in the HR-Admin group can post announcements; department channels are visible only to members of the corresponding Cognito group.

5. Message Search

Index the messages table in Amazon OpenSearch Service to enable full-text search across channels and DM history, surfaced through a search bar on the frontend.